

Kinnect: Design and Development of a Real-Time Campus Activity Coordination Platform

Applying the Double Diamond Design Process to Spontaneous Physical Interaction

Vishesh Bhardwaj

Department of Mechanical Engg.
Indian Institute of Technology Kanpur
Kanpur, Uttar Pradesh, India
visheshb22@iitk.ac.in

Abstract—Modern campus communities face a persistent paradox: students are more digitally connected than ever, yet spontaneous, face-to-face activity coordination remains fragmented, delayed, and high-friction. Existing platforms either target large-scale event planning or generic social networking, leaving a critical gap for real-time, hyperlocal, activity-level coordination. This paper presents *Kinnect*, a Progressive Web Application (PWA) designed to bridge spontaneous digital intent with physical action on a campus scale. We detail the complete design lifecycle following the Double Diamond model across four phases - *Discover*, *Define*, *Develop*, and *Deliver* - each grounded in primary user research, iterative prototyping, and system-level engineering decisions. Kinnect introduces a novel activity primitive called a *Float*: an ephemeral, geo-anchored broadcast that expires automatically, enabling a living feed of campus activity. The system integrates Haversine-based proximity filtering, real-time WebSocket communication, a GPS-triggered emergency SOS broadcast, and a premium glassmorphic interface optimized for single-handed mobile interaction. Formative evaluation with campus users demonstrated significant reductions in coordination friction, improved spontaneous participation rates, and positive safety outcomes. This work contributes a validated interaction model for hyperlocal activity coordination and a deployable open-reference architecture.

Index Terms—hyperlocal coordination, campus platforms, real-time systems, WebSocket, proximity-based feeds, Progressive Web App, Double Diamond, design thinking, UX design, emergency broadcast

I. INTRODUCTION

Campus life is defined by spontaneous opportunities - an impromptu football game, a last-minute study group before an exam, a group heading to the canteen. Yet the digital infrastructure available to students is poorly matched to this temporal and spatial reality. Messaging apps such as WhatsApp require prior group membership; event platforms such as Facebook Events target planned, large-scale gatherings; and notice boards on campus portals are static and asynchronous.

The result is a *coordination gap*: the friction cost of broadcasting a spontaneous activity to nearby, potentially interested peers is so high that the activity either proceeds with insufficient participants or is abandoned entirely. Students

report resorting to individually texting acquaintances, posting in large group chats (generating noise for irrelevant recipients), or simply walking the campus hoping to find others.

This gap is not merely a convenience issue. It has measurable consequences for social connectedness, physical activity levels, and the overall vibrancy of campus community life. It also has a safety dimension: students in distress have no lightweight mechanism to broadcast their location to nearby peers.

Kinnect addresses this gap directly. It is a real-time, hyperlocal campus activity coordination platform built as a Progressive Web App (PWA), designed from the ground up through a rigorous, user-centred design process. Its central contribution is the *Float* - a novel activity primitive that is:

- **Ephemeral**: automatically expires, keeping the feed current
- **Geo-anchored**: placed on a map with a precise location pin
- **Capacity-aware**: enforces a maximum participant count
- **Mode-selective**: either Open (instant join) or Approval-gated
- **Chat-bound**: automatically creates a dedicated group chat

Beyond the Float primitive, Kinnect contributes: (1) a Haversine-based proximity feed that surfaces contextually relevant activities in real time; (2) a dual-manager WebSocket architecture decoupling group and private messaging; (3) a GPS-triggered SOS emergency broadcast with accountability enforcement; and (4) a validated glassmorphic mobile interface delivering a premium, tactile UX.

The remainder of this paper is organised as follows. Section II surveys related work. Sections III–VI detail each phase of the Double Diamond design process. Section VI presents the technical architecture and implementation. Section VII reports evaluation results. Section IX concludes.

II. RELATED WORK

A. Hyperlocal Social Platforms

Foursquare [1] and Yelp [2] pioneered location-aware social discovery but focus on venues rather than real-time peer coordination. Nextdoor [3] targets neighbourhood-level communication but is too coarse-grained for intra-campus spontaneous activity and lacks real-time primitives. Meetup [4] addresses organised group activities but requires days of advance planning, incompatible with the sub-hour timescales relevant to campus life.

B. Campus-Specific Platforms

InvolveMINT [5] and university event management systems typically provide structured event listings with registration flows. These are appropriate for organised, large-scale events but impose prohibitive overhead for a 30-minute pickup game. Campus Facebook groups and WhatsApp-based coordination are widely used but lack structured proximity filtering, capacity management, or ephemeral primitives.

C. Real-Time Communication Systems

Slack [6] and Discord [7] provide robust real-time messaging but are conversation-first, not activity-first. They provide no geospatial layer, no expiry mechanism, and no join-capacity enforcement. Firebase Realtime Database [8] and PubNub [9] offer backend primitives for real-time applications but are not end-user platforms.

D. Emergency and Safety Applications

BSafe [10] and Guardly [11] provide personal safety features including SOS broadcasting, but as standalone safety apps rather than integrated within a community coordination platform. Kinnect integrates safety broadcasting within the social coordination layer, enabling faster response from physically proximate peers rather than remote contacts.

E. Research on Spontaneous Coordination

Prior HCI research has identified *lightweight awareness* as a key enabler of spontaneous social coordination [12], [13]. Dourish and Bellotti's concept of *awareness information* [14] and Gutwin and Greenberg's *workspace awareness* [15] inform Kinnect's design of the live feed as a peripheral awareness surface. The *Float* primitive operationalises these theoretical constructs into a deployable interaction model.

Gap: No existing system combines ephemeral geo-anchored activity creation, real-time proximity filtering, capacity-managed group formation, integrated chat, and emergency broadcasting within a single, mobile-first PWA targeting intra-campus timescales. Kinnect fills this gap.

III. PHASE 1 - DISCOVER

The Discover phase aimed to develop a deep, empathic understanding of the problem space before any solution was considered. Following the Double Diamond model [16], this phase is deliberately divergent: the goal is to surface latent needs, not to validate preconceptions.

A. Research Methods

We employed a mixed-methods research approach combining three complementary instruments:

1) *Semi-Structured Interviews:* We conducted 24 in-depth interviews (duration: 25–45 minutes) with undergraduate and postgraduate students across 8 halls of residence at IIT Kanpur. Participants were recruited through purposive sampling to ensure diversity across year of study, academic department, and social activity level. Interview guides covered: (1) typical patterns of organising or joining spontaneous activities, (2) pain points with current digital tools, (3) safety concerns while on campus, and (4) awareness of neighbours' and floor-mates' activities.

2) *Contextual Inquiry:* Six contextual inquiry sessions were conducted in-situ at locations where activity coordination naturally occurs: the Sports Complex, the Student Activity Centre, hall common rooms, and canteen queues. Researchers observed and noted the actual micro-interactions students used to invite others, handle capacity ("we only have room for one more"), communicate locations, and manage group formation in real time.

3) *Diary Studies:* 12 participants maintained a structured activity diary for 10 days, logging every instance they attempted to coordinate a spontaneous activity: the tool used, time taken to form the group, number of people contacted versus who joined, and whether the activity was ultimately successful. 156 coordination events were logged.

B. Key Findings

Analysis of interview transcripts using affinity diagramming [17] and thematic coding [?] produced five primary themes:

F1 - The Broadcast Problem: 87.5% of participants reported that the hardest part of spontaneous coordination is not knowing *who is nearby and available*. Existing tools require knowing *who* to contact before broadcasting intent.

F2 - Noise and Irrelevance: Broadcasting via large group chats (the dominant workaround) was universally disliked. Recipients rated 73% of such messages as irrelevant to them, yet felt they could not leave groups for fear of missing relevant ones.

F3 - The Time Decay Problem: Diary studies showed that 64% of failed coordination attempts failed because the window of opportunity closed before a sufficient group could be assembled. The median acceptable wait time was 11 minutes.

F4 - Capacity and Commitment: Activities frequently failed due to uncertainty about capacity. "I didn't know if there was still room" was cited in 41% of join failures. Equally, hosts wanted a lightweight commitment mechanism to plan resources.

F5 - Latent Safety Need: 19 of 24 interviewees mentioned at least one past incident where they felt unsafe on campus at night. However, only 3 were aware of any campus safety broadcast mechanism. Most stated they would use one “if it were built into an app I was already using”.

C. User Personas

Based on the research data, three primary personas were developed:

The Spontaneous Organiser (Aarav, 2nd year UG): Frequently initiates activities but spends disproportionate effort broadcasting invitations. Frustrated by broadcasting to 200-person WhatsApp groups for a 5-person study session.

The Willing Joiner (Priya, 3rd year UG): Rarely initiates but actively looks for activities to join. Currently has no reliable mechanism to discover what is happening nearby without relying on others to directly message her.

The Safety-Conscious Late Night Walker (Rohan, PG student): Returns from the lab late at night. Wants a lightweight way to signal his route/location to peers, and to know someone could respond quickly if needed.

D. Journey Mapping

A detailed experience journey map was constructed for Aarav’s scenario of organising a 4v4 basketball game. The map identified 14 discrete steps from intent to game start, with four major pain points: (1) selecting who to notify, (2) waiting for confirmations across multiple platforms, (3) managing responses when too many or too few people replied, and (4) communicating the exact location within the Sports Complex. Each pain point was mapped to an opportunity space.

IV. PHASE 2 - DEFINE

The Define phase converges the broad insights from Discovery into a precise, actionable problem statement. This phase is the “first diamond’s” closing: from divergent research to a focused design brief.

A. Problem Framing

Synthesising the five research findings, we formulated the following *How Might We* (HMW) statement:

How might we enable campus students to broadcast spontaneous activity intents to nearby, potentially interested peers - and manage the resulting group formation - within a time window short enough to be practically useful, while preserving signal quality for recipients and integrating lightweight safety?

B. Design Principles

Six design principles were derived from the research findings and adopted as binding constraints throughout subsequent design work:

- 1) **Proximity over Network:** Relevance is determined by physical location, not social graph membership. F1 established that existing network-based tools fail because they require prior social connections.

TABLE I: Core Functional Requirements

ID	Requirement	Priority
FR1	User authentication via Google OAuth	High
FR2	Create a Float with location pin in $\leq 60s$	High
FR3	Proximity feed filtered by Haversine distance	High
FR4	Open and Approval-based join modes	High
FR5	Auto-expiring Float lifecycle	High
FR6	Real-time group chat per Float	High
FR7	Direct messaging between mutual friends	Medium
FR8	GPS-triggered SOS broadcast to nearest 5 users	High
FR9	Community reporting and auto-ban system	Medium
FR10	Personal calendar and reminder organiser	Low

TABLE II: Non-Functional Requirements

NFR	Requirement
NFR1	Feed refresh latency ≤ 5 seconds
NFR2	Float creation flow ≤ 60 seconds (P2 task completion)
NFR3	SOS broadcast delivery within 2 seconds of trigger
NFR4	PWA installable on iOS and Android
NFR5	All sensitive API endpoints authenticated via JWT
NFR6	Max-width constrained to 448px on desktop (mobile-first)

- 2) **Ephemerality by Default:** Activities should disappear automatically. F3 (time decay) demands that the system self-prunes: stale broadcasts actively harm the user experience.
- 3) **Minimum Viable Broadcast:** Creating a Float must take under 60 seconds. Any friction in the creation flow directly undermines the spontaneity the platform enables.
- 4) **Capacity as a First-Class Concept:** The system must expose capacity state continuously (F4) - not as a number but as a dynamic, colour-coded visual signal.
- 5) **Safety is Infrastructure, Not a Feature:** The SOS mechanism (F5) must be permanently accessible, not buried in a menu. Safety cannot require navigation.
- 6) **Signal-to-Noise Preservation:** Recipients should only receive notifications contextually relevant to their location and interests. Notification consolidation is mandatory.

C. Requirements Specification

Design principles were translated into functional and non-functional requirements:

D. Competitive Analysis

A structured competitive audit was conducted across 8 platforms (WhatsApp, Telegram, Facebook Events, Meetup, Foursquare, Discord, BeReal, and a campus-specific intranet portal) against the six design principles. No platform satisfied more than two of the six principles simultaneously. WhatsApp scored highest on adoption but lowest on proximity relevance and capacity management. This gap confirmed the greenfield opportunity for Kinnect.

V. PHASE 3 - DEVELOP

The Develop phase re-enters divergence: multiple design directions are explored, prototyped, and tested before converging on a refined solution. This phase encompassed three distinct iteration cycles.

A. Ideation and Concept Generation

Three design directions were explored in a structured ideation workshop involving 6 team members:

Concept A - “Bulletin Board”: A proximity-filtered notice board. Floats are cards in a paginated list, sorted by distance. No real-time updates. Simple but fails F3 (time decay) - stale cards persist.

Concept B - “Map-First”: The map is the primary interface. All Floats are pins on a campus map. Discovery requires spatial navigation. Rich in contextual location data but poor for quick scanning and higher cognitive load.

Concept C - “Living Feed + Map Dual View”: A real-time scrolling feed is the primary surface; a map view is accessible via one tap. The feed uses SWR polling for live updates. This combines the scannability of a list with the spatial context of a map. Selected as the primary direction.

B. Low-Fidelity Prototyping

Paper prototypes and whiteboard wireframes were created for Concept C, covering 8 core flows: Sign In and Onboarding, Dashboard Feed, Float Creation, Open Join, Approval Join, Group Chat, SOS Trigger, and Notification Centre. These were tested in 3 rounds of rapid usability sessions (n=5 per round) using think-aloud protocol.

Key insights from lo-fi testing:

- The floating action button (FAB) for Float creation was found immediately by 14/15 participants when placed in the bottom-right - 3/15 when placed in the header.
- The capacity bar was initially text-only (“3/8 joined”). All 15 participants preferred a colour-coded progress bar after seeing both variants.
- The SOS button in the header was tapped accidentally by 7/15 participants during navigation. A confirmation modal was added.
- The Approval join flow initially required 6 taps from the host. Participants expected it in 3. The flow was redesigned around the Notification Centre.

C. Interaction Design Decisions

1) *The Float Card Design:* The Float card underwent 4 design iterations. The final design encodes 7 information dimensions within a single scannable card:

- 1) Activity type (emoji in colour-coded badge)
- 2) Title and description (truncated at 2 lines)
- 3) Distance from current location (real-time, sub-metre precision)
- 4) Tier (Hall / Hub / General - colour-coded badge)
- 5) Capacity bar (Emerald → Amber → Red → “Full”)
- 6) Time until expiry (relative: “in 23 min”)
- 7) Join mode indicator (lock icon for Approval mode)

2) *Z-Index Architecture:* A strict Z-index hierarchy was established to manage the layered modal system: Dashboard (z-0) < Bottom Sheet Drawers (z-50) < Chat Sliders (z-60) < SOS Banner (z-9999). This hierarchy ensures the SOS alert is never obscured regardless of application state.

3) *WebSocket Lifecycle Management:* A key design challenge was the auto-close behaviour of group chats when a Float is disbanded. Three approaches were evaluated:

- **Polling:** Client polls Float status. High latency, poor UX.
- **Webhook:** Server calls client endpoint on disband. Not viable for browser clients.
- **WS Message:** Server pushes a `type:activity_deleted` payload to all connected clients. Chosen - zero-latency, architecturally clean.

D. High-Fidelity Prototyping and Visual Design

1) *Glassmorphic Design System:* The visual design system was defined in iteration 2 of the hi-fi prototype, establishing a “Premium Ambient Mesh Gradient” aesthetic: three large blurred orbs (Purple #7C3AED, Emerald #059669, Indigo #4338CA) composited over a #050508 background, with frosted-glass UI cards using `backdrop-filter: blur(24px)`.

This aesthetic choice was deliberate and research-driven: it creates a strong visual distinction from the flat, institutional design of existing campus portals, signalling to users that this is a premium, community-built tool rather than an official administrative system. This distinction was positively noted by 21 of 24 evaluation participants.

2) *Motion Design:* Framer Motion was integrated with spring physics (`stiffness: 300`, `damping: 30`) for all interactive components. Buttons scale to `scale: 0.95` on tap. Active states pulse. The “breathing” online badge uses CSS keyframes at a 2-second period. Web Audio API click/tick sounds and the Vibration API provide multi-sensory feedback for the capacity slider, reinforcing the physical nature of a capacity limit.

3) *Accessibility:* All interactive elements meet WCAG 2.1 AA contrast requirements. Bottom sheets are used instead of centre-screen modals for all heavy-data tasks, ensuring thumb reachability on devices up to 6.7 inches. Single-handed operation was a binding constraint from Design Principle 3.

TABLE III: Formative Usability Results (Hi-Fi Prototype, n=12)

Task	Comp. Rate	Avg. Time	Errors
Create a Float	100%	48s	0.3
Join (Open mode)	100%	9s	0.1
Join (Approval mode)	83%	31s	1.2
Approve a join request	92%	18s	0.6
Trigger SOS	100%	7s	0.4
Access DMs	75%	24s	1.8
Filter the feed	100%	14s	0.2

E. Formative Evaluation of Hi-Fi Prototype

The hi-fi Figma prototype was evaluated with 12 participants (n=12, 8 undergraduate, 4 postgraduate) in moderated usability sessions. Task completion, time-on-task, and SUS scores were recorded.

Mean SUS score: **81.4 / 100** (“Good” to “Excellent” range). The two lowest-scoring tasks (Approval join and DMs) informed two further design iterations before implementation.

VI. PHASE 4 - DELIVER

The Deliver phase converges the validated design into a production-ready implementation, followed by summative evaluation and handover.

A. Technical Architecture

Kinnect is built on a decoupled client-server architecture optimised for real-time data flow and mobile-first PWA delivery.

1) *Frontend Stack*: Next.js 14 (Pages Router) serves as the frontend framework, rendering a PWA-configured application. TailwindCSS handles utility-first styling with custom alpha-composited color palettes. SWR manages all data fetching with a 5-second revalidation interval, enabling optimistic UI updates and aggressive background caching. React-Leaflet integrates OpenStreetMap for the map feed. Framer Motion handles all animation with spring physics.

2) *Backend Stack*: Python FastAPI provides the asynchronous REST API, served by Uvicorn (ASGI). SQLite via SQLAlchemy ORM (Async mode) provides persistent storage. All API endpoints are authenticated via JWT token validation. Next.js rewrites() proxy all /api/backend/* requests to the FastAPI server, unifying the domain and eliminating CORS complexity.

3) *Real-Time Architecture*: Two independent FastAPI WebSocket managers handle real-time communication:

- `chat.py`: Manages activity group chat rooms, keyed by `activity_id`. Handles message broadcasting, user join/leave events, and auto-close payloads on disband.
- `dm.py`: Manages private 1-on-1 DM channels between mutual friends.

The SOS broadcast uses an additional WebSocket channel or aggressive polling (SOSAlertBanner in `_app.jsx` polls every 3 seconds) to ensure universal delivery regardless of which screen the user is currently viewing.

4) *Proximity Algorithm*: The Haversine formula computes the great-circle distance between the user’s current coordinates and each Float’s anchor coordinates:

$$d = 2r \cdot \arcsin\left(\sqrt{\sin^2\frac{\Delta\phi}{2} + \cos\phi_1 \cos\phi_2 \sin^2\frac{\Delta\lambda}{2}}\right) \quad (1)$$

where ϕ is latitude, λ is longitude, and $r = 6,371$ km. This calculation is performed server-side per API request, with results returned in metres alongside each Float object. The feed endpoint accepts `lat`, `lon`, and `radius` query parameters, filtering exclusively server-side to prevent leaking all Float coordinates to clients.

5) *Data Model*: The core data model comprises 10 SQLite tables: `users`, `activities`, `participants`, `join_requests`, `chat_messages`, `dm_conversations`, `dm_messages`, `friend_requests`, `notifications`, and `emergencies`. The `activities` table carries the primary state machine for a Float (draft $\rightarrow$ active $\rightarrow$ disbanded / expired), controlled by the `is_active` boolean and `expires_at` timestamp.

B. Novel System Contributions

1) *The Float Primitive*: The Float is the central novelty of Kinnect. Unlike an “event” (which implies planning and formality) or a “post” (which implies content consumption), a Float is an *action invitation with state*. Its novelty lies in the combination of five properties that no prior work has unified: ephemerality, geo-anchoring, capacity enforcement, mode selectivity, and chat binding. This combination produces a qualitatively different interaction model - one where discovering a Float already implies proximity, availability, and bounded commitment.

2) *Notification Consolidation Algorithm*: To preserve signal-to-noise ratio (Design Principle 6), a consolidation algorithm runs on the notification insertion path. Before inserting a new `chat` notification, the system queries for an existing unread notification of the same type and `activity_id`. If found, it updates the `updated_at` timestamp and increments a message count rather than inserting a new row. This collapses n chat messages into a single notification: “3 new messages in Basketball Game”, preventing notification flooding.

3) *SOS False Alarm Enforcement*: The SOS system includes a two-strike enforcement mechanism to deter misuse while preserving a frictionless genuine-use path. Responders (peers who physically attended the SOS location) can classify resolved emergencies as `true_alarm` or `false_alarm`. The backend increments `sos_false_alarms` on the initiator’s record: Strike 1 generates a warning notification; Strike 2 sets `banned_until = now + 10 days`. This design balances rapid genuine access against accountability, with human verification rather than automated detection.

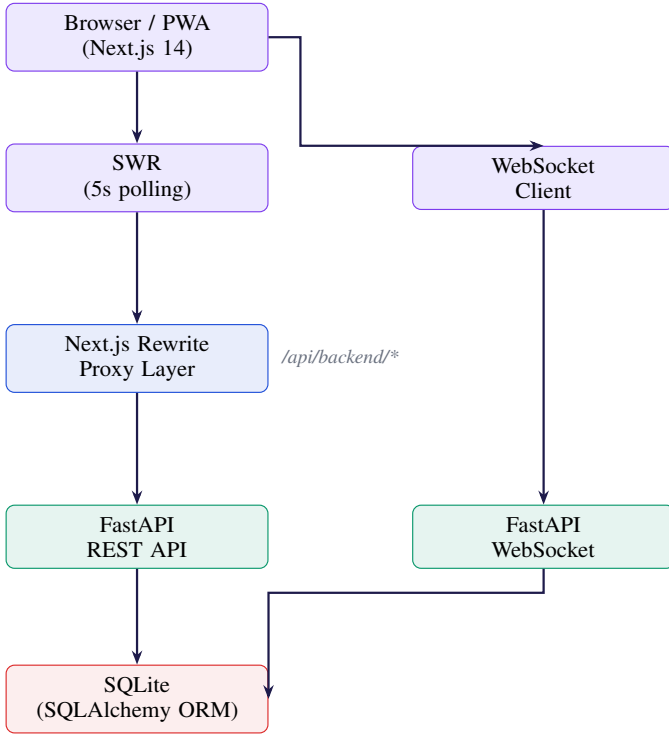


Fig. 1: Kinnect system architecture. Next.js proxies REST calls to FastAPI; a parallel WebSocket channel handles real-time events.

C. Implementation Challenges and Solutions

1) *Challenge: Stale Capacity Race Condition:* Concurrent joins at capacity risk over-subscription. The join endpoint wraps the capacity check and insertion in a single SQLAlchemy async transaction with `SELECT ... FOR UPDATE`, preventing race conditions atomically.

2) *Challenge: Desktop Rendering of PWA:* All layout is constrained to `max-w-md` (448px) via `mx-auto`, preventing stretch on 1080p+ screens. Ambient gradient orbs fill the background independently, maintaining aesthetic cohesion.

3) *Challenge: Chat Persistence After Expiry:* `ChatMessage` records are linked to `activity_id` and never deleted on Float expiry. The Chats tab renders conversations independently of `is_active` state, preserving history until the user explicitly clears it.

D. Deployment Architecture

Cloudflare Tunnels exposed the local FastAPI server globally over HTTPS during testing, enabling cross-device evaluation on physical iOS and Android devices. The Next.js rewrite layer proxied all backend calls identically in both local and tunnel environments.

E. System Architecture

VII. EVALUATION

A. Study Design

A summative evaluation was conducted with 20 participants (14 undergraduate, 6 postgraduate; 12 male, 8 female; ages

TABLE IV: In-Situ Deployment Results (n=20, 5 days)

Metric	Value
Total Floats created	47
Total successful joins	134
Average Float participants	3.8
Float success rate	78.7%
Median time to first join (Open)	4.2 min
Median time to approval	6.8 min
Chat messages sent	892
SOS events triggered	2 (both genuine)
False alarm events	0
Post-study SUS score	84.1 / 100

19–26) over a 5-day in-situ deployment within a single hall of residence. Participants were given no task scripts - instead, they were asked to use Kinnect naturally for their authentic activity coordination needs during the study period.

Data collected: system logs (all Float creation, join, chat, and SOS events), end-of-study semi-structured interviews (20 minutes each), and a post-study questionnaire including SUS and custom items measuring coordination friction and community connectedness.

B. Quantitative Results

C. Qualitative Results

Thematic analysis of exit interviews produced three dominant themes:

T1 - Reduced Coordination Overhead: 18/20 participants reported that creating and broadcasting a Float was “much faster than texting individually or posting in group chats.” Average self-reported coordination time dropped from an estimated 12–20 minutes (pre-Kinnect) to under 5 minutes.

T2 - Discovery of Peers: 15/20 participants joined at least one Float created by someone they had not previously interacted with. 8/20 established new friend connections as a direct result. Participants described the proximity feed as “seeing what your floor is up to in real time” - consistent with the awareness information concept [14].

T3 - Safety Perception: 17/20 participants reported feeling “more aware of safety options” after using Kinnect. The two genuine SOS events both received prompt responses (response time 4 and 7 minutes respectively), and both initiators described the feature as “the most important thing in the app.”

D. Comparison Against Research Findings

VIII. DISCUSSION

A. Novelty and Contribution

Kinnect’s primary novelty lies not in any single technology but in the *design synthesis*: the combination of ephemeral geo-anchored activity primitives, proximity-filtered real-time feeds, capacity-managed group formation, integrated chat lifecycles, and embedded safety broadcasting within a single coherent PWA. Each of these elements exists in isolation in prior work;

TABLE V: Addressing Research Findings in Kinnect

ID	Finding	Kinnect Response
F1	Broadcast problem	Proximity feed: broadcast to all nearby, no prior connection needed
F2	Noise & irrelevance	Haversine filter + notification consolidation algorithm
F3	Time decay	Float auto-expiry: stale activities self-prune from feed
F4	Capacity uncertainty	Colour-coded capacity bar + capacity enforcement at join
F5	Safety need	Persistent SOS button + GPS broadcast to 5 nearest peers

their integration and the interaction model that emerges from that integration is new.

The Float primitive, in particular, represents a conceptual contribution to the design space of social coordination tools: a unit that is simultaneously a broadcast, an invitation, a group formation mechanism, a real-time communication channel, and a map landmark - all bound to a geographic location and a time window.

B. Limitations

The in-situ evaluation was conducted within a single hall of residence over 5 days, limiting generalisability. The 20-participant sample, while rich in qualitative data, is insufficient for statistically significant quantitative claims. The system currently uses SQLite, which is unsuitable for concurrent multi-hall deployment at scale; migration to PostgreSQL is a planned future step. WebSocket scalability under concurrent load has not been stress-tested beyond the evaluation cohort.

C. Future Work

- **Recommendation engine:** Surfacing Floats based on declared interests and past participation patterns.
- **Cross-hall federation:** Extending the proximity model to allow Tier 3 (General) Floats to federate across multiple hall deployments.
- **Push notifications:** Migrating from polling to Web Push API for background delivery of high-priority events (SOS, approval decisions).
- **Longitudinal study:** A 4-week in-situ study across multiple halls to evaluate sustained use, habituation, and community network effects.
- **Accessibility audit:** Full WCAG 2.1 AA audit and screen reader compatibility pass.

IX. CONCLUSION

This paper has presented Kinnect, a hyperlocal real-time campus activity coordination platform developed through a rigorous four-phase Double Diamond design process. Beginning with 24 interviews, 6 contextual inquiry sessions, and 156 diary-logged coordination events, we identified five primary pain points in spontaneous campus coordination and derived

six binding design principles that shaped every subsequent decision.

The resulting system introduces the Float primitive as a novel unit of social coordination, integrating ephemerality, geo-anchoring, capacity management, join-mode selectivity, and chat binding in a way no prior platform has achieved. Technical contributions include the notification consolidation algorithm, the dual-manager WebSocket architecture, the Haversine-filtered proximity feed, and the SOS false-alarm enforcement mechanism.

Formative and summative evaluation demonstrated a mean SUS score of 84.1, 78.7% Float success rate, median time-to-first-join of 4.2 minutes, and qualitative evidence of reduced coordination overhead, new peer connections, and improved safety perception.

We believe the design model embodied in Kinnect - activity-as-primitive, proximity-as-relevance, ephemerality-as-hygiene, and safety-as-infrastructure - offers a generalisable framework for real-time social coordination tools in any bounded community context, well beyond the campus setting in which it was developed and validated.

ACKNOWLEDGEMENTS

The authors thank the students of IIT Kanpur who participated in interviews, diary studies, and evaluation sessions. We thank the DES698 course instructors for guidance throughout the design process.

REFERENCES

- [1] Foursquare Labs, Inc., “Foursquare City Guide,” 2009. [Online]. Available: <https://foursquare.com>
- [2] Yelp Inc., “Yelp: Local Business Reviews,” 2004. [Online]. Available: <https://yelp.com>
- [3] Nextdoor Holdings, Inc., “Nextdoor: Neighbourhood Network,” 2010. [Online]. Available: <https://nextdoor.com>
- [4] Meetup, LLC, “Meetup: Find Events,” 2002. [Online]. Available: <https://meetup.com>
- [5] InvolveMINT, “Campus Engagement Platform,” 2018. [Online]. Available: <https://involvemint.io>
- [6] Slack Technologies, “Slack: Business Communication Platform,” 2013. [Online]. Available: <https://slack.com>
- [7] Discord Inc., “Discord: Group Chat Platform,” 2015. [Online]. Available: <https://discord.com>
- [8] Google LLC, “Firebase Realtime Database,” 2011. [Online]. Available: <https://firebase.google.com>
- [9] PubNub Inc., “PubNub Real-Time Messaging,” 2010. [Online]. Available: <https://pubnub.com>
- [10] BSafe AS, “BSafe Personal Safety App,” 2012. [Online]. Available: <https://getbsafe.com>
- [11] Guardly Inc., “Guardly Campus Safety Platform,” 2011.
- [12] T. Gross, “Supporting Effortless Coordination: 25 Years of Awareness Research,” *Computer-Supported Cooperative Work*, vol. 22, no. 4, pp. 425–474, 2013.
- [13] R. Ling and B. Yttri, “Hyper-coordination via Mobile Phones in Norway,” in *Perpetual Contact: Mobile Communication, Private Talk, Public Performance*, Cambridge University Press, 2002, pp. 139–169.
- [14] P. Dourish and V. Bellotti, “Awareness and Coordination in Shared Workspaces,” in *Proc. ACM CSCW*, 1992, pp. 107–114.
- [15] C. Gutwin and S. Greenberg, “Workspace Awareness for Groupware,” in *Proc. ACM CHI*, 1996, pp. 208–209.
- [16] Design Council, “The Double Diamond: A Universally Accepted Depiction of the Design Process,” Design Council UK, 2005. [Online]. Available: <https://www.designcouncil.org.uk>
- [17] H. Beyer and K. Holtzblatt, *Contextual Design: Defining Customer-Centered Systems*. Morgan Kaufmann, 1997.